CS598_PSL_PA_1

November 9, 2026

0.1 CS 598 PSL Fall 2026

0.1.1 Project 1 Support Vector Machines and a Comparative Study of Classifiers

- Anirudh Eswara
- Ryan Lenea
- Mark Synowiec
- Robert Walker

[1]:
```
import pandas as pd

RANDOM_SEED = 598

df = pd.read_csv("kepler_koi.csv")

# only physical transit/stellar columns; excludes kepoi_name (ID) and␣
 ↪koi_disposition (response)
predictor_cols = [
    "koi_period", "koi_duration", "koi_depth", "koi_impact", "koi_model_snr",
    "koi_prad", "koi_teq", "koi_insol", "koi_sma", "koi_incl", "koi_ror",
    "koi_steff", "koi_slogg", "koi_srad", "koi_smass", "koi_smet",
    "koi_kepmag", "koi_srho",
]

X = df[predictor_cols].copy()
y = df["koi_disposition"].copy()

print(X.shape)
print(y.value_counts())
X.head()
```

```
(7587, 18)
koi_disposition
FALSE POSITIVE    4839
CONFIRMED         2748
Name: count, dtype: int64
```

[1]:

	koi_period	koi_duration	koi_depth	koi_impact	koi_model_snr	koi_prad	\
0	2.470613	1.74319	14230.9	0.818	4304.3	13.04	

```
1    2.204735       3.88864     6674.7       0.224         5945.9     16.10
2    4.887803       2.36255     4315.8       0.054          862.3      4.82
3    3.849372       2.63200     1298.6       0.915          132.7     12.94
4    3.213669       3.98235      727.8       0.021          346.9      4.13

   koi_teq  koi_insol  koi_sma  koi_incl   koi_ror  koi_steff  koi_slogg  \
0   1339.0     761.46   0.0354     84.41  0.123871     5820.0      4.457
1   2048.0    4148.92   0.0376     87.21  0.075522     6440.0      4.019
2    800.0      96.67   0.0530     89.82  0.057823     4778.0      4.591
3   2034.0    4055.29   0.0548     80.61  0.039601     6244.0      3.657
4   1515.0    1244.44   0.0440     89.81  0.024470     5795.0      4.101

   koi_srad  koi_smass  koi_smet  koi_kepmag  koi_srho
0     0.964      0.971     -0.06      11.338   1.83538
1     1.952      1.449      0.14      10.463   0.37936
2     0.763      0.830      0.32       9.174   3.69348
3     2.992      1.482     -0.16      11.432   0.22463
4     1.545      1.097      0.18      12.211   0.46728
```

0.2 Scientific Question

The scientific question is whether the transit geometry and the host-star properties, by themselves, distinguish confirmed planets from false positives, and which classifier from the course you would trust for that triage.

0.3 Setup

Shared train/test split and missing-value imputation used by every method below.

[2]:
```python
from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    train_size=0.8,
    stratify=y, # stratify as required
    random_state=RANDOM_SEED,
)

print(X_train.shape, X_test.shape)
print(y_train.value_counts(normalize=True))
print(y_test.value_counts(normalize=True))

# impute missing physical measurements using training-set medians only
imputer = SimpleImputer(strategy="median")
X_train_imp = imputer.fit_transform(X_train)
X_test_imp = imputer.transform(X_test)
```

```
(6069, 18) (1518, 18)
koi_disposition
FALSE POSITIVE    0.637832
CONFIRMED         0.362168
Name: proportion, dtype: float64
koi_disposition
FALSE POSITIVE    0.637681
CONFIRMED         0.362319
Name: proportion, dtype: float64
```

0.3.1 (i) LDA

[3]:
```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.metrics import accuracy_score

lda = LinearDiscriminantAnalysis()
lda.fit(X_train_imp, y_train)

y_train_pred_lda = lda.predict(X_train_imp)
y_test_pred_lda = lda.predict(X_test_imp)

train_acc_lda = accuracy_score(y_train, y_train_pred_lda)
test_acc_lda = accuracy_score(y_test, y_test_pred_lda)

print(f"LDA train accuracy: {train_acc_lda:.4f}")
print(f"LDA test accuracy: {test_acc_lda:.4f}")
print("Test confusion matrix (rows=true, cols=predicted):")
print(pd.crosstab(y_test, y_test_pred_lda, rownames=["True"],␣
  ↪colnames=["Predicted"]))
```

```
LDA train accuracy: 0.8192
LDA test accuracy: 0.8182
Test confusion matrix (rows=true, cols=predicted):
Predicted       CONFIRMED  FALSE POSITIVE
True
CONFIRMED             428             122
FALSE POSITIVE        154             814
```

0.3.2 (ii) Logistic Regression

[4]:
```
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# unpenalized logistic regression in the original units
# newton-cholesky handles the unscaled better than lbfgs
# https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
logreg = LogisticRegression(
```

```
    solver="newton-cholesky",
    max_iter=5000,
    C=np.inf
)
logreg.fit(X_train_imp, y_train)

y_train_pred_logreg = logreg.predict(X_train_imp)
y_test_pred_logreg = logreg.predict(X_test_imp)

train_acc_logreg = accuracy_score(y_train, y_train_pred_logreg)
test_acc_logreg = accuracy_score(y_test, y_test_pred_logreg)

print(f"Logistic regression train accuracy: {train_acc_logreg:.4f}")
print(f"Logistic regression test accuracy: {test_acc_logreg:.4f}")
print("Test confusion matrix (rows=true, cols=predicted):")
print(pd.crosstab(y_test, y_test_pred_logreg, rownames=["True"],␣
 ↪colnames=["Predicted"]))
```

```
Logistic regression train accuracy: 0.8585
Logistic regression test accuracy: 0.8696
Test confusion matrix (rows=true, cols=predicted):
Predicted       CONFIRMED  FALSE POSITIVE
True
CONFIRMED             486              64
FALSE POSITIVE        134             834
```

0.3.3 (iii) Linear SVM

[5]:
```
### (iii) Linear SVM
from sklearn.svm import SVC
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import GridSearchCV, StratifiedKFold
from sklearn.metrics import accuracy_score

linsvm_grid = GridSearchCV(
    make_pipeline(StandardScaler(), SVC(kernel="linear")),
    {"svc__C": [0.001, 0.01, 0.1, 1, 10, 100, 1000]},
    cv=StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_SEED),
    n_jobs=-1,
)
linsvm_grid.fit(X_train_imp, y_train)
linsvm = linsvm_grid.best_estimator_
print("Best params:", linsvm_grid.best_params_)

y_train_pred_linsvm = linsvm.predict(X_train_imp)
y_test_pred_linsvm = linsvm.predict(X_test_imp)
```

```
train_acc_linsvm = accuracy_score(y_train, y_train_pred_linsvm)
test_acc_linsvm = accuracy_score(y_test, y_test_pred_linsvm)

print(f"Linear SVM train accuracy: {train_acc_linsvm:.4f}")
print(f"Linear SVM test accuracy: {test_acc_linsvm:.4f}")
print("Test confusion matrix (rows=true, cols=predicted):")
print(pd.crosstab(y_test, y_test_pred_linsvm, rownames=["True"],␣
 ↪colnames=["Predicted"]))
```

```
Best params: {'svc__C': 1000}
Linear SVM train accuracy: 0.8568
Linear SVM test accuracy: 0.8729
Test confusion matrix (rows=true, cols=predicted):
Predicted       CONFIRMED  FALSE POSITIVE
True
CONFIRMED             494              56
FALSE POSITIVE        137             831
```

0.3.4 (iv) Kernel SVM

[6]:
```
from sklearn.svm import SVC
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import GridSearchCV, StratifiedKFold
from sklearn.metrics import accuracy_score

# RBF kernel uses distances, so scale (scaler is fit on training folds only via␣
 ↪the pipeline)
# 5-fold stratified CV on the training set tunes C and gamma; test set is not␣
 ↪touched
svm_pipe = make_pipeline(StandardScaler(), SVC(kernel="rbf"))
svm_grid = GridSearchCV(
    svm_pipe,
    {
        "svc__C": [0.1, 1, 10, 100, 1000],
        "svc__gamma": [0.001, 0.01, 0.1, 1, 10],
    },
    cv=StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_SEED),
    n_jobs=-1,
    return_train_score=True,
)
svm_grid.fit(X_train_imp, y_train)
ksvm = svm_grid.best_estimator_
print("Best params:", svm_grid.best_params_)

y_train_pred_ksvm = ksvm.predict(X_train_imp)
```

```
y_test_pred_ksvm = ksvm.predict(X_test_imp)

train_acc_ksvm = accuracy_score(y_train, y_train_pred_ksvm)
test_acc_ksvm = accuracy_score(y_test, y_test_pred_ksvm)

print(f"Kernel SVM train accuracy: {train_acc_ksvm:.4f}")
print(f"Kernel SVM test accuracy: {test_acc_ksvm:.4f}")
print("Test confusion matrix (rows=true, cols=predicted):")
print(pd.crosstab(y_test, y_test_pred_ksvm, rownames=["True"],␣
 ↪colnames=["Predicted"]))

# Overfit check
res = pd.DataFrame(svm_grid.cv_results_)
res["gap"] = res["mean_train_score"] - res["mean_test_score"]
print(res[["param_svc__C", "param_svc__gamma", "mean_train_score",␣
 ↪"mean_test_score", "gap"]]
      .sort_values(["param_svc__gamma", "param_svc__C"]).round(4).
 ↪to_string(index=False))
```

```
Best params: {'svc__C': 1000, 'svc__gamma': 0.01}
Kernel SVM train accuracy: 0.9043
Kernel SVM test accuracy: 0.8913
Test confusion matrix (rows=true, cols=predicted):
Predicted       CONFIRMED  FALSE POSITIVE
True
CONFIRMED             502              48
FALSE POSITIVE        117             851
 param_svc__C  param_svc__gamma  mean_train_score  mean_test_score     gap
          0.1             0.001            0.6378           0.6378 -0.0000
          1.0             0.001            0.8171           0.8176 -0.0005
         10.0             0.001            0.8348           0.8341  0.0007
        100.0             0.001            0.8504           0.8487  0.0016
       1000.0             0.001            0.8731           0.8695  0.0036
          0.1             0.010            0.8195           0.8201 -0.0005
          1.0             0.010            0.8389           0.8362  0.0026
         10.0             0.010            0.8618           0.8588  0.0030
        100.0             0.010            0.8858           0.8789  0.0069
       1000.0             0.010            0.9041           0.8917  0.0124
          0.1             0.100            0.8428           0.8382  0.0046
          1.0             0.100            0.8730           0.8652  0.0078
         10.0             0.100            0.9024           0.8880  0.0144
        100.0             0.100            0.9277           0.8913  0.0365
       1000.0             0.100            0.9495           0.8896  0.0599
          0.1             1.000            0.8491           0.8384  0.0108
          1.0             1.000            0.9254           0.8827  0.0428
         10.0             1.000            0.9714           0.8796  0.0919
        100.0             1.000            0.9920           0.8665  0.1254
```

1000.0	1.000	0.9997	0.8535	0.1462
0.1	10.000	0.6394	0.6385	0.0009
1.0	10.000	0.9967	0.7696	0.2271
10.0	10.000	1.0000	0.7899	0.2101
100.0	10.000	1.0000	0.7898	0.2102
1000.0	10.000	1.0000	0.7898	0.2102

0.3.5 (v) Additional Method - Random Forrest

[7]:
```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GridSearchCV, StratifiedKFold
from sklearn.metrics import accuracy_score

forest_grid = GridSearchCV(
    RandomForestClassifier(oob_score=True, random_state=RANDOM_SEED),
    {
        "max_features": ["sqrt", "log2", None]
    },
    cv=StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_SEED),
    n_jobs=-1,
    return_train_score=True,
)
forest_grid.fit(X_train_imp, y_train)
forest = forest_grid.best_estimator_
print("Best params:", forest_grid.best_params_)

y_train_pred_forest = forest.predict(X_train_imp)
y_test_pred_forest = forest.predict(X_test_imp)

train_acc_forest = accuracy_score(y_train, y_train_pred_forest)
test_acc_forest = accuracy_score(y_test, y_test_pred_forest)

print(f"Random Forest Classifier train accuracy: {train_acc_forest:.4f}")
print(f"Random Forest Classifier OOB accuracy: {forest.oob_score_:.4f}")
print(f"Random Forest Classifier test accuracy: {test_acc_forest:.4f}")
print("Test confusion matrix (rows=true, cols=predicted):")
print(pd.crosstab(y_test, y_test_pred_forest, rownames=["True"],␣
 ↪colnames=["Predicted"]))
```

```
Best params: {'max_features': 'sqrt'}
Random Forest Classifier train accuracy: 1.0000
Random Forest Classifier OOB accuracy: 0.9255
Random Forest Classifier test accuracy: 0.9321
Test confusion matrix (rows=true, cols=predicted):
Predicted       CONFIRMED  FALSE POSITIVE
True
CONFIRMED             502              48
FALSE POSITIVE         55             913
```

0.3.6 Chosen Classifier

The Random Forest Classifier was picked because it can capture nonlinear relationships and interactions among features.

0.4 Report Results

###(a) Protocol

- **Seed:** 598 for the split, the CV folds, and the forest.
- **Split:** 80% train / 20% test, stratified by `koi_disposition`.
- **Missing values:** "median imputation", with medians computed on the training set only.
- **Scaling:** `StandardScaler` (training folds only, inside the pipeline) for both SVMs. LDA, unpenalized logistic regression, and the forest use the original units.
- **Tuning:** 5-fold stratified CV on the training set only. Linear SVM (hinge loss, L2): C in {0.001, 0.01, 0.1, 1, 10, 100, 1000}. Kernel SVM (RBF): C in {0.1, 1, 10, 100, 1000} and gamma in {0.001, 0.01, 0.1, 1, 10}. Forest: *max_features* in {sqrt, log2, None}.
- **Priors and costs:** default training-set class frequencies; error costs are not changed.

0.4.1 (b) Comparison

[8]:
```
acc_data = {
    "Training Accuracy" : [train_acc_lda, train_acc_logreg, train_acc_linsvm,␣
 ↪train_acc_ksvm, train_acc_forest],
    "Test Accuracy": [test_acc_lda, test_acc_logreg, test_acc_linsvm,␣
 ↪test_acc_ksvm, test_acc_forest]
}

accuracy_table = pd.DataFrame(acc_data, index=["LDA", "Logistic Regression",␣
 ↪"Linear SVM", "Kernel SVM", "Random Forest Classifier"])
print(accuracy_table)
print()

def summarize(method, y_t, y_pred):
    print("----------------------")
    print(f"{method}:")
    print(f"Test confusion matrix (rows=true, cols=predicted):")
    ct = pd.crosstab(y_t, y_pred, rownames=["True"], colnames=["Predicted"])
    print(ct)
    tp = ct.iloc[1, 1]
    fn = ct.iloc[0, 1]
    fp = ct.iloc[1, 0]
    tn = ct.iloc[0, 0]
    print()
    print(f"Confirmed Planets called False Positives {ct.iloc[0,1]}")
    print(f"False Positives called Planets: {ct.iloc[1,0]}")
    print(f"Precision: {round(tp / (tp + fp), 3)} Recall: {round(tp / (tp +␣
 ↪fn),3)}")
```

```
    print()

summarize("LDA", y_test, y_test_pred_lda)

summarize("Logistic Regression", y_test, y_test_pred_logreg)

summarize("Linear SVM", y_test, y_test_pred_linsvm)

summarize("Kernel SVM", y_test, y_test_pred_ksvm)

summarize("Random Forest", y_test, y_test_pred_forest)
```

```
                          Training Accuracy  Test Accuracy
LDA                                0.819245       0.818182
Logistic Regression                0.858461       0.869565
Linear SVM                         0.856813       0.872859
Kernel SVM                         0.904268       0.891304
Random Forest Classifier           1.000000       0.932148

----------------------
LDA:
Test confusion matrix (rows=true, cols=predicted):
Predicted       CONFIRMED  FALSE POSITIVE
True
CONFIRMED             428             122
FALSE POSITIVE        154             814

Confirmed Planets called False Positives 122
False Positives called Planets: 154
Precision: 0.841 Recall: 0.87

----------------------
Logistic Regression:
Test confusion matrix (rows=true, cols=predicted):
Predicted       CONFIRMED  FALSE POSITIVE
True
CONFIRMED             486              64
FALSE POSITIVE        134             834

Confirmed Planets called False Positives 64
False Positives called Planets: 134
Precision: 0.862 Recall: 0.929

----------------------
Linear SVM:
Test confusion matrix (rows=true, cols=predicted):
Predicted       CONFIRMED  FALSE POSITIVE
True
```

```
CONFIRMED             494              56
FALSE POSITIVE        137             831

Confirmed Planets called False Positives 56
False Positives called Planets: 137
Precision: 0.858 Recall: 0.937

----------------------
Kernel SVM:
Test confusion matrix (rows=true, cols=predicted):
Predicted       CONFIRMED  FALSE POSITIVE
True
CONFIRMED             502              48
FALSE POSITIVE        117             851

Confirmed Planets called False Positives 48
False Positives called Planets: 117
Precision: 0.879 Recall: 0.947

----------------------
Random Forest:
Test confusion matrix (rows=true, cols=predicted):
Predicted       CONFIRMED  FALSE POSITIVE
True
CONFIRMED             502              48
FALSE POSITIVE         55             913

Confirmed Planets called False Positives 48
False Positives called Planets: 55
Precision: 0.943 Recall: 0.95
```

0.4.2 (c) Figure

[9]:
```
import matplotlib.pyplot as plt

conf = df[df["koi_disposition"] == "CONFIRMED"]
fp = df[df["koi_disposition"] == "FALSE POSITIVE"]
wrong = X_test[y_test_pred_forest != y_test.values]

plt.figure(figsize=(10, 6))
plt.scatter(fp["koi_period"], fp["koi_prad"], alpha=0.3, label="FALSE POSITIVE")
plt.scatter(conf["koi_period"], conf["koi_prad"], alpha=0.3, label="CONFIRMED")
plt.scatter(wrong["koi_period"], wrong["koi_prad"], marker="x", color="black",␣
 ↪label="Forest test errors")
plt.xscale("log")
plt.yscale("log")
```

```
plt.xlabel("Orbital Period (days)")
plt.ylabel("Planet Radius (Earth radii)")
plt.title("Orbital Period vs. Planet Radius")
plt.legend()
plt.show()
```

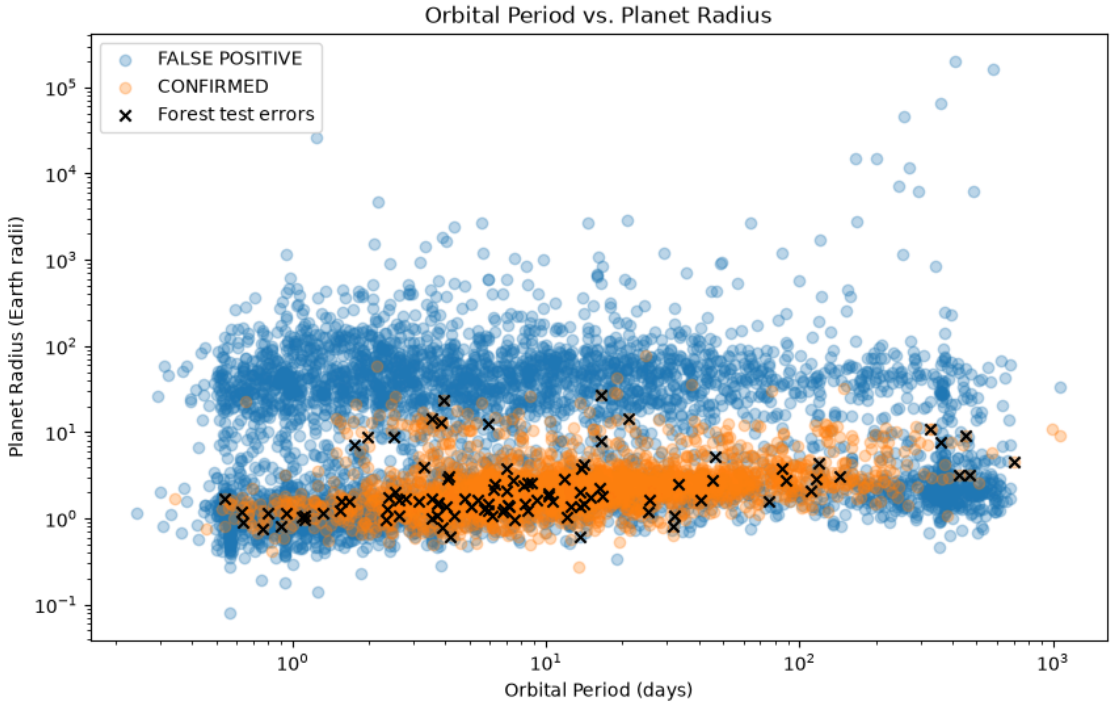


Figure 1: KOI: Orbital Period vs Planet Radius. Confirmed planets cluster at 1 to 15 Earth radii, mostly 1 to 4, with periods of roughly 1 to 100 days. False positives have a second band at 20 to 1000 Earth radii, which may be star-sized objects rather than planets. We plotted the random forest's test errors on top of this plot to answer the question: Given the 93 percent accuracy, where did the errors land? We can see that all 103 errors fall below about 25 Earth radii, in the same radius and period range as the confirmed planets. 40 percent of the errors fall between 3 and 10 days, the range where the data are closest to half planets and half false positives. This figure is evidence that the random forest lines up with intuition: Its misses are objects that look like real planets when taking into account orbital period and planetary radius.

0.4.3 (d) Discussion

Which method would you hand an observer who has to decide which KOIs get followup spectra, and why? Test accuracy alone is not a complete answer. Which error is worse in this application, and did your ranking of methods change when you looked at the two error types separately?

The method that I would hand to the observer would be the Random Forest Classifier method. I would hand this method to the observer because when evaluated on the test dataset, it is tied with

the Kernel SVM for the smallest number of confirmed planets called false positives and had the smallest number of false positives called planets. In this application confirmed planets called false positives would be worse since real planets would miss out on the followup spectra. Although false positives called planets is also undesired, the observer could simply perform the followup spectra and determine that the initial prediction that the KOI was a planet was incorrect. If we rank the five methods on accuracy which treats the two errors equally we get the following ranking 1) Random Forest Classifier 2) Kernel SVM 3) Linear SVM 4) Logistic Regression 5) LDA. If we rank the five methods on confirmed planets called false positives we get the following ranking 1) Random Forest Classifier and Kernel SVM 2) Linear SVM 3) Logistic Regression 4) LDA. If we rank the five methods on false positives called planets we get the following ranking 1) Random Forest Classifier 2) Kernel SVM 3) Logistic Regression 4) Linear SVM 5) LDA.

Linear SVM versus logistic regression: do they disagree on many test objects? Relate this to hinge loss versus logistic loss (points beyond the margin; a probability model versus a score).

[10]:
```
import numpy as np

disagreement_indices = np.where(y_test_pred_linsvm != y_test_pred_logreg)
print("Indices of Test Objects where Linear SVM and Logistic Regression␣
 ↪Disagree:", disagreement_indices[0])
print("Number of Test Objects where Linear SVM and Logistic Regression Disagree:
 ↪", len(disagreement_indices[0]))
print("Percentage of Test Objects where Linear SVM and Logistic Regression␣
 ↪Disagree:", len(disagreement_indices[0]) / len(X_test_imp))

X_test_imp_disagreement = X_test_imp[disagreement_indices]
linsvm_scores = linsvm.decision_function(X_test_imp_disagreement)
logreg_scores = logreg.decision_function(X_test_imp_disagreement)
logreg_prob = logreg.predict_proba(X_test_imp_disagreement)[:,1]

num_label = np.where(y_test.iloc[disagreement_indices] == "FALSE POSITIVE", 1,␣
 ↪-1)
df = pd.DataFrame({
    "True Label": y_test.iloc[disagreement_indices],
    "Linear SVM Prediction": y_test_pred_linsvm[disagreement_indices],
    "Logistic Regression Prediction": y_test_pred_logreg[disagreement_indices],
    "Linear SVM Score": linsvm_scores,
    "Logistic Regression Score": logreg_scores,
    "Logistic Regression Probability": logreg_prob,
    "Hinge Loss": np.maximum(0,1 - num_label * linsvm_scores),
    "Logistic Loss": np.logaddexp(0, -num_label * logreg_scores)
})
df
```

```
Indices of Test Objects where Linear SVM and Logistic Regression Disagree: [  72
186  195  241  242  355  356  358  391  462  582  608  612  624
  694  793  829  830  834  895  924  945 1004 1028 1194 1211 1212 1376
```

1403 1479 1513]
Number of Test Objects where Linear SVM and Logistic Regression Disagree: 31
Percentage of Test Objects where Linear SVM and Logistic Regression Disagree:
0.020421607378129116

[10]:

	True Label	Linear SVM Prediction	Logistic Regression Prediction
2434	CONFIRMED	CONFIRMED	FALSE POSITIVE
2191	CONFIRMED	FALSE POSITIVE	CONFIRMED
6322	FALSE POSITIVE	FALSE POSITIVE	CONFIRMED
6814	FALSE POSITIVE	FALSE POSITIVE	CONFIRMED
5759	FALSE POSITIVE	FALSE POSITIVE	CONFIRMED
5019	FALSE POSITIVE	CONFIRMED	FALSE POSITIVE
1004	CONFIRMED	CONFIRMED	FALSE POSITIVE
1829	CONFIRMED	FALSE POSITIVE	CONFIRMED
89	CONFIRMED	CONFIRMED	FALSE POSITIVE
6395	FALSE POSITIVE	FALSE POSITIVE	CONFIRMED
4730	FALSE POSITIVE	CONFIRMED	FALSE POSITIVE
6748	FALSE POSITIVE	CONFIRMED	FALSE POSITIVE
833	CONFIRMED	CONFIRMED	FALSE POSITIVE
340	CONFIRMED	CONFIRMED	FALSE POSITIVE
3402	FALSE POSITIVE	CONFIRMED	FALSE POSITIVE
1330	CONFIRMED	CONFIRMED	FALSE POSITIVE
1800	CONFIRMED	CONFIRMED	FALSE POSITIVE
3690	FALSE POSITIVE	CONFIRMED	FALSE POSITIVE
3435	FALSE POSITIVE	FALSE POSITIVE	CONFIRMED
7299	FALSE POSITIVE	CONFIRMED	FALSE POSITIVE
2538	CONFIRMED	CONFIRMED	FALSE POSITIVE
2804	FALSE POSITIVE	CONFIRMED	FALSE POSITIVE
2119	CONFIRMED	CONFIRMED	FALSE POSITIVE
2603	CONFIRMED	FALSE POSITIVE	CONFIRMED
6197	FALSE POSITIVE	FALSE POSITIVE	CONFIRMED
374	CONFIRMED	CONFIRMED	FALSE POSITIVE
6405	FALSE POSITIVE	CONFIRMED	FALSE POSITIVE
3102	FALSE POSITIVE	FALSE POSITIVE	CONFIRMED
2931	FALSE POSITIVE	CONFIRMED	FALSE POSITIVE
2230	CONFIRMED	CONFIRMED	FALSE POSITIVE
3700	FALSE POSITIVE	CONFIRMED	FALSE POSITIVE

	Linear SVM Score	Logistic Regression Score
2434	-0.047778	0.052420
2191	0.036082	-0.009636
6322	0.441654	-0.166094
6814	0.162109	-0.362479
5759	0.072115	-0.001312
5019	-0.032445	0.206294
1004	-0.029451	0.178259
1829	0.122975	-0.011682

89	-0.026358	0.075318
6395	0.340442	-0.224726
4730	-0.204494	0.038604
6748	-0.586467	0.205730
833	-0.156710	0.016167
340	-0.107396	0.164957
3402	-0.118443	0.483380
1330	-0.104220	0.220971
1800	-0.051645	0.217176
3690	-0.112235	0.102837
3435	0.106535	-0.050584
7299	-0.215768	0.034725
2538	-0.008316	0.359313
2804	-0.138111	0.161033
2119	-0.195576	0.124768
2603	0.099486	-0.047123
6197	0.290014	-0.025222
374	-0.219871	0.268863
6405	-0.310341	0.297814
3102	0.245508	-0.256669
2931	-0.043801	0.042603
2230	-0.135261	0.034729
3700	-0.082585	0.135986

	Logistic Regression Probability	Hinge Loss	Logistic Loss
2434	0.513102	0.952222	0.719701
2191	0.497591	1.036082	0.688341
6322	0.458572	0.558346	0.779639
6814	0.410360	0.837891	0.890722
5759	0.499672	0.927885	0.693803
5019	0.551391	1.032445	0.595311
1004	0.544447	0.970549	0.786244
1829	0.497080	1.122975	0.687323
89	0.518821	0.973642	0.731515
6395	0.444054	0.659558	0.811810
4730	0.509650	1.204494	0.674031
6748	0.551252	1.586467	0.595564
833	0.504042	0.843290	0.701263
340	0.541146	0.892604	0.779023
3402	0.618546	1.118443	0.480384
1330	0.555019	0.895780	0.809724
1800	0.554082	0.948355	0.807619
3690	0.525687	1.112235	0.643050
3435	0.487357	0.893465	0.718759
7299	0.508680	1.215768	0.675935
2538	0.588874	0.991684	0.888856
2804	0.540172	1.138111	0.615868

2119	0.531152	0.804424	0.757476
2603	0.488221	1.099486	0.669863
6197	0.493695	0.709986	0.705838
374	0.566814	0.780129	0.836587
6405	0.573908	1.310341	0.555286
3102	0.436183	0.754492	0.829694
2931	0.510649	1.043801	0.672073
2230	0.508681	0.864739	0.710662
3700	0.533944	1.082585	0.627464

The disagreements between the predictions of the the Linear SVM and Logistic Regression models are due to the fact that they are optimized with different loss functions with Linear SVM using Hinge Loss and Logistic Regression using Logistic Loss. Hinge Loss tries to maximize the margin between the two classes and correctly classified observations that lie beyond the margin are not penalized of zero whereas correctly classified observations that lie within the margin or or are incorrecly classified are penalizws. On the other hand Logisitic Loss penalizes observations based on their predicted probability which causes all observations to have some penalty.

Also Linear SVM is usually interpreted by observing the output of the decision function whereas the Logistic Regression is interpreted by observing the predicted proabilities. It is notable that the observations for which the two models disagreed, the Linear SVM model's decision function produced values between -1 and 1 and the Logistic Regression model predicted probabiities between 0.4 and 0.6. Both models indicate that these observations are close to their respective classification boundaries and are difficulty to classify which explains why the models produced different predictions.

Does the kernel SVM improve on the linear SVM enough to justify the extra tuning? Does a very large C look like overfit?

We improve by ~2% from 0.87 to 0.89 acuracy using our prefered SVM (C = 1000, gamma = 0.01) over linear, which feel like a worthwhile improvement for the limited additional complexity and time (about 30 seconds extra of runtime on a M3 Macbook). When gamma AND C are both large, we do overfit. The worst case was when C was 1000 and gamma was 10, which resulted in perfect (remembered) acuracy on the testing and a train/test difference of 21%.

SVM scores are not b P(CONFIRMED | x). If the observer wanted a "needs more data" bin instead of a hard call, which of your methods is actually built for that? Say what you would do instead of treating sign(f(x)) as a posterior.

Logistic regression is the method built for a "needs more data" bin. It models P(CONFIRMED | x) directly and is fit by maximizing the likelihood, so its outputs can be read as probabilities. We could hand the observer three groups: objects with a predicted probability of being a planet above 0.7 are treated as confirmed planets, objects below 0.3 are treated as false positives, and objects between 0.3 and 0.7 go into the "needs more data" bin. The SVM's f(x) is a signed distance from the separating hyperplane, not a probability. Hinge loss ignores points beyond the margin, so the SVM is never trained to make the size of f(x) match how likely an object is to be a planet, and sign(f(x)) only gives confirmed planet versus false positive for our study. However, we could train a logistic regression to learn from the output of SVM. The logistic regression could be fit to the SVM score and planet label as output. That fitted score from logistic regression is a probability, and we could apply the same 0.3 and 0.7 cutoffs to sort objects into confirmed, false positive, and

needs more data.

One limitation of the study (missingness that is not at random, confirmed planets being easier targets, a single split, residual label leakage if you slipped a flag into X, ...) and how it should change how strongly you believe the ranking of methods.

A large limiting factor of this study is nonrandom missing data. 281 of the 7,587 rows have at least one missing measurement (3.7%). Of the rows with a missing measurement, 279 are false positives (5.8%) and 2 are confirmed planets (0.07%). This changes how we strongly we believe the predictive capability of the random forest. The nature of random forests gives the model a prediction advantage against other models with nonrandom missing data. Median imputation gives every missing measurement the same value. A deep tree can isolate that exact value with two nearby splits, so the forest may be learning that a measurement was not recorded rather than learning from given data. The kernel SVM is less able to exploit this pattern: with gamma = 0.01 its boundary is smooth, so it cannot isolate one exact imputed value the way a deep tree can, although imputed rows still sit close together and may be partly learned as a cluster. But does the difference in accuracy between the highest accuracy model, Random Forest and the second highest accuracy model, the Kernel SVM, come solely from this data discrepancy? 3.7% of the test data has missing measurements, and the difference in accuracy between the two (Kernel SVM at 0.891, Random Forest at 0.932) is 4.1%. Since the difference in accuracy between the random forest and the kernel svm is higher than the percentage of rows with missing data, and although the random forest learns from missing data better than the kernel svm, we still rank the random forest as the best model.